Milestone 2.2: Classification

INFO 521 · Machine Learning Foundations · project milestone brief · From Inference to Discovery · Gate A (Supervised) · 2 of 2

> **How to read this**
> This is the assignment brief for one project milestone. The Satisfactory criteria below are authoritative: every one must be met, and it is all-or-nothing rather than partial credit. A Not-Yet returns with targeted feedback and one revise cycle. Do the work in the notebook distributed through Classroom 50; this PDF is the brief, not the workspace.

Course-schedule placement. Part 2 of the project: begins at the midpoint, due Week 7.5. This milestone covers M6 and is due Week 6. Gated by Gate A, with 2.1 (Checkpoint quiz 2.1).

The margin lens

In 2.1 you predicted hypertension with a *probability* and reasoned about its posterior. Now change the question from "how probable?" to "**which side of the boundary, with the widest possible margin?**", a **support vector machine**. It is *discriminative* and *geometric*: no likelihood, no posterior, just a separating hyperplane chosen to sit as far as possible from both classes. You will build a soft-margin SVM from scratch, score it honestly, derive its dual, and then stage the real contest: the probabilistic classifier of 2.1 versus this margin machine, on the **same split**.

Ground rules. Implement every estimator and metric from scratch in NumPy/SciPy. No scikit-learn, no off-the-shelf SVM, QP solver, or metric function. The `info521` library gives you data, plotting, and self-checks only. Your own prior work is reusable as always: the 2.1 split and standardization helpers carry over unchanged. The SVM and the metrics are new builds (no homework unit rehearses them; this notebook scaffolds them).

Leakage rule (same as 2.1). Features are the **six non-BP columns** `ds["X"]`; the label is `hypertension(ds)`. **Never** use `sbp`/`dbp` (or any function of them) as a feature. Fit standardization on the **training split only**.

What "Satisfactory" means for 2.2

- ☐ You implement a **linear soft-margin SVM** from scratch (primal **hinge loss + L_2 penalty**, optimized by **subgradient / Pegasos-style descent**) and explain the role of C and of the **margin**.
- ☐ You implement **precision, recall, and F1** from scratch and evaluate on a **held-out** split.
- ☐ You **derive the dual** in markdown and explain the **kernel trick** and *why* one kernelizes (conceptual; a kernelized implementation is an **optional stretch**, not required for Satisfactory).
- ☐ You **compare** the 2.1 probabilistic classifier against the SVM on the **same train/test split**: when does each win, and what does each *express*?

- ☐ **Play artifact** (below) complete and interpreted.

Setup

```
import numpy as np
from info521 import data, plotting, checks

plotting.use_course_style()
ds = data.load_clinical()

X_raw = ds["X"]                        # (n, 6): age, bmi, waist, chol, hdl, hba1c
feature_names = ds["features"]
labels, rule = data.hypertension(ds)  # leakage-safe ACC/AHA label in {0, 1}

print(f"{X_raw.shape[0]} patients · {X_raw.shape[1]} features = {feature_names}")
print(f"label = {rule}")
```

0 · Reuse the split; map labels to ±1

Use the **same** seeded train/test split and the **train-only** standardization from 2.1 (copy those helpers in, or import your own module). The SVM convention is $t_n \in \{-1, +1\}$, so map the $\{0, 1\}$ hypertension label accordingly; keep the original $\{0, 1\}$ version around for your metrics.

```
# TODO: rebuild X_train, X_test, y_train, y_test exactly as in 2.1 (same seed),
# standardize with TRAIN-only mean/sd, and form t_train, t_test in {-1, +1}.
```

1 · Soft-margin SVM (primal, from scratch)

Minimize the regularized hinge objective

$$J(\mathbf{w}, b) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{n=1}^{N} \max\!\left(0,\, 1 - t_n(\mathbf{w}^\top \mathbf{x}_n + b)\right) .$$

The margin has width $2/\|\mathbf{w}\|$; the hinge term pays a linear price for points inside the margin or misclassified. C sets the exchange rate: large $C \Rightarrow$ violations are expensive $\Rightarrow$ a narrow, hard margin; small $C \Rightarrow$ a wide, forgiving margin. Optimize with a **subgradient** method (the hinge is non-smooth); the Pegasos schedule (step size $\propto 1/t$) is a clean choice.

```
def svm_fit(X, t, C=1.0, n_epochs=100, seed=521):
    # TODO: subgradient / Pegasos descent on J(w, b). Return (w, b). At each step
    # the hinge subgradient is 0 for correctly-classified-with-margin points and
    # -t_n x_n otherwise; add the L2 term's gradient. Track J to show it decreasing.
    raise NotImplementedError

def svm_decision(X, w, b):
    # TODO: return the signed score w·x + b (NOT yet thresholded).
    raise NotImplementedError

def svm_predict(X, w, b):
    # TODO: return hard labels in {-1, +1} = sign of the decision score.
    raise NotImplementedError
```

2 · Precision, recall, F1 (from scratch)

For the positive (hypertensive) class, with TP/FP/FN counted on the **held-out** split:

$$\text{precision} = \frac{TP}{TP+FP}, \quad \text{recall} = \frac{TP}{TP+FN}, \quad F_1 = \frac{2\,\mathrm{P}\,\mathrm{R}}{\mathrm{P}+\mathrm{R}}.$$

```
def precision_recall_f1(y_true01, y_pred01):
    # TODO: count TP/FP/FN from {0,1} arrays; return (precision, recall, f1).
    # Decide and DOCUMENT how you handle the degenerate zero-denominator cases.
    raise NotImplementedError

# TODO: evaluate the SVM on the test split. Report precision, recall, F1: not just
# accuracy (the classes are ~60/40, so accuracy alone can mislead).
```

What to expect, modest, but real. These six non-BP features carry only weak signal about a blood-pressure-defined label, so a correctly built classifier reaches only modest discrimination here (AUC in the mid-0.6s) and an accuracy near the majority-class baseline (~0.61). That is **above chance but far from strong**, it is the honest result, not a bug. Don't tune yourself into believing otherwise; report the precision–recall picture and read the modest performance as a real finding about how much these features can say about hypertension.

3 · The dual and the kernel trick (markdown)

In a markdown cell, **derive the dual** of the soft-margin SVM via Lagrange multipliers $\alpha_n \geq 0$ on the margin constraints (this matches the course's high-level Lagrange/duality treatment, you need the *result* and its meaning, not a KKT proof):

$$\max_{\boldsymbol{\alpha}} \sum_n \alpha_n - \frac{1}{2}\sum_n \sum_m \alpha_n \alpha_m t_n t_m \, \mathbf{x}_n^\top \mathbf{x}_m \quad \text{s.t.} \quad 0 \leq \alpha_n \leq C, \; \sum_n \alpha_n t_n = 0.$$

Then explain, in prose:

- **Support vectors**: why only points on/inside the margin have $\alpha_n > 0$.
- **The kernel trick**: the data enter *only* through inner products $\mathbf{x}_n^\top \mathbf{x}_m$, so replacing them with a kernel $k(\mathbf{x}_n, \mathbf{x}_m)$ fits a linear boundary in an implicit high-dimensional feature space (a **nonlinear** boundary in the original space) without ever forming that space.
- **Why kernelize here**: is a linear boundary plausibly enough for these six features, or would an RBF kernel likely help? Argue from what you saw in 2.1.

***** Optional stretch (not required for Satisfactory)

Implement a **kernelized** SVM (e.g. RBF) by optimizing the dual, and compare its test F1 to the linear model. Keep it clearly separated from the required work.

4 · Probabilistic vs margin, same split

```
# TODO: bring your 2.1 MAP/posterior classifier and this SVM together on the SAME
# test split. Tabulate precision/recall/F1 for both. For the probabilistic model,
# also report something the SVM cannot give: a calibrated probability / an
# uncertainty on the prediction.
```

Write 200–300 words: when does each win? The SVM commits to a hard max-margin boundary; the Bayesian logistic model returns a *probability* with *uncertainty*. For which clinical uses does a calibrated probability matter more than a crisp decision, and when is the opposite true?

Play artifact: the threshold sweep & the C knob

1. **Threshold sweep (probabilistic model).** Sweep the decision threshold of your 2.1 classifier from 0 to 1; at each threshold compute precision and recall and trace the **precision–recall tradeoff curve**. Mark the default 0.5 point. The SVM gives one (precision, recall) point unless you also threshold its score, contrast that.
2. **The C knob (SVM).** Vary C across several orders of magnitude. Watch the **margin width** ($2/\|\mathbf{w}\|$), the **number of support vectors** (points with non-zero hinge), and the **train/test errors** move together.

Write 150–250 words reading both: how the threshold trades precision against recall *for a fixed model*, versus how C trades margin width against training violations *by changing the model*.

```
# TODO: the PR-tradeoff curve + the C sweep (margin, #SVs, errors) + interpretation.
```

> **i** Gate A
>
> 2.1 and 2.2 together form the **supervised gate**. Per `GRADING.md`, Gate A is complete when **both** are Satisfactory **and Checkpoint 2.1** (the Newton–Raphson derivation gating 2.1) is passed. There is no separate checkpoint for 2.2.

Looking ahead

Gate B drops the labels entirely. In **2.3** you let **k-means** discover structure in the same six-feature space with no access to the hypertension label, then check whether the unsupervised subgroups *recover* the clinical strata anyway.